

Machine Learning Assignment 2

Roll Number: 23004 | Seed: 4

This report documents from-scratch implementations of k-Means Clustering, Gaussian Naive Bayes, and a C4.5 Decision Tree. All datasets are programmatically generated using seed 4 (last 3 digits of roll number 23004). Scikit-learn is used only for train-test splitting and evaluation metrics.

1. Dataset Construction

Nine datasets were generated across three categories. Each satisfies the minimum requirements of $n \geq 1000$ samples and $d \geq 15$ features, with at least 5 informative and at least 5 noisy features. The high-dimensional dataset has $d = 60$ features and $n = 5000$ samples, meeting the special requirements.

Dataset	n	d	Informative	Noise	Purpose
Low Noise	1500	15	5	10	Decision Tree, NB baseline
High Noise	2000	20	5	15	Model robustness
High Dimensional	5000	60	10	50	Feature selection test
k-Means Friendly	1500	15	2 (spatial)	13	k-Means success
k-Means Adversarial	500	15	2 (spatial)	13	k-Means failure
Correlated Features	2000	15	3	10	NB violation
NB Works (violation)	1500	15	3	12	NB robustness
NB Fails (rings)	2000	15	2 (spatial)	13	NB decision boundary
Greedy Counterexample	1200	15	2 (structured)	13	DT greedy analysis

1.1 Assumption Analysis

Low-noise dataset: Features are linearly separable with minimal label noise. Assumptions satisfied: Gaussian features, informative signals dominate. Expected: low bias, good generalisation for all three algorithms.

High-noise dataset: 25% label flipping and high-variance noise features. Violated: stable decision boundary assumption. Trees will overfit noisy labels; NB will be over-confident; k-Means centroids will drift.

High-dimensional dataset: 50 of 60 features are pure Gaussian noise. Violated: feature relevance assumption for IG. C4.5 Gain Ratio penalises high-cardinality spurious splits, so it handles this better than ID3. k-Means suffers from the curse of dimensionality.

2. k-Means Clustering

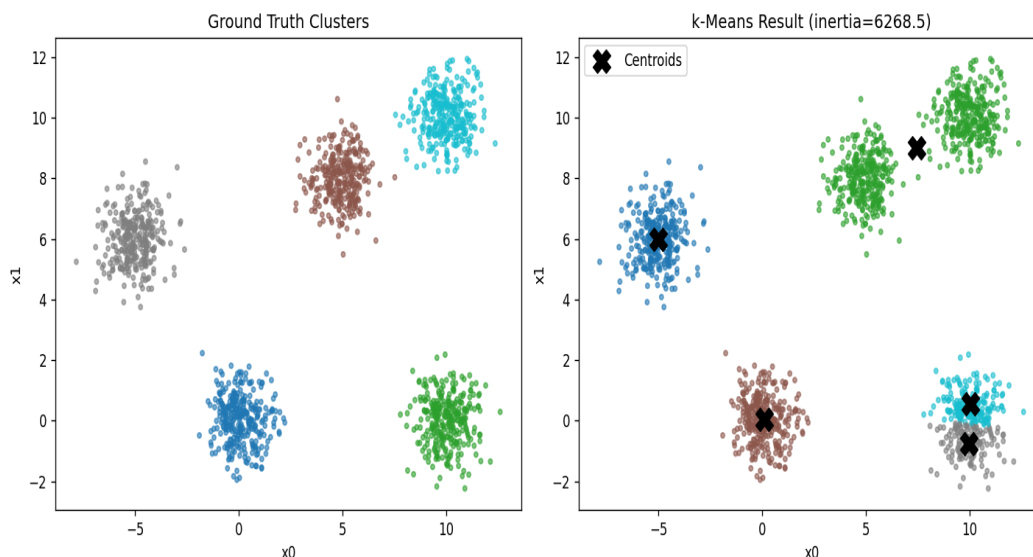
2.1 Implementation

The implementation uses fully vectorised NumPy operations. Centroid initialisation randomly selects k distinct data points. Distance computation uses broadcasting: $X[:, \text{newaxis}, :] - \text{centroids}[\text{newaxis}, :, :]$ produces an (n, k, d) tensor from which Euclidean norms are computed along axis 2. Convergence is detected when the L2 norm of centroid shifts falls below $\text{tol}=1\text{e-}4$.

No Python-level loops are used for the inner distance or assignment steps. Only the outer iteration loop and the centroid update (which requires per-cluster means) use list comprehensions.

2.2 Friendly Dataset

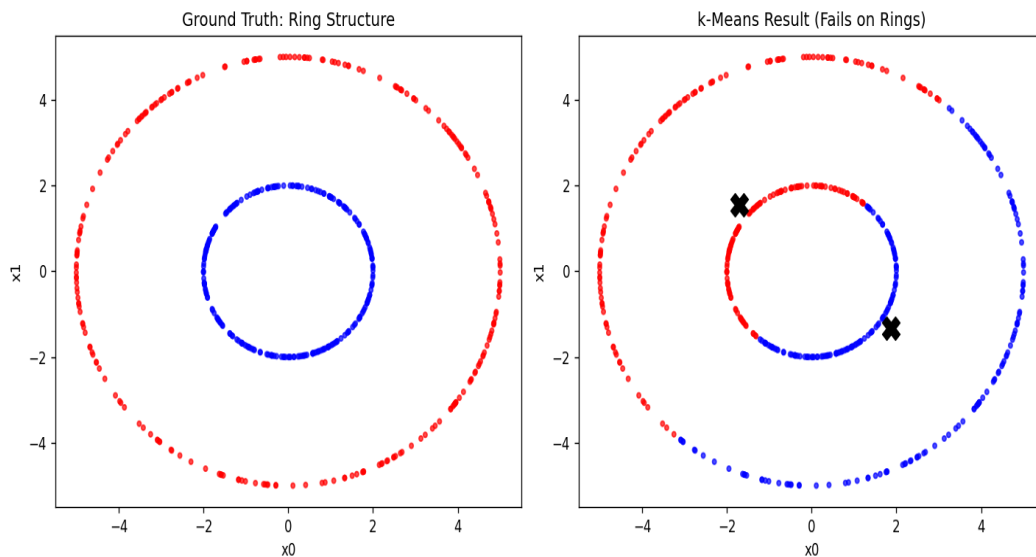
Five Gaussian clusters are well-separated in 2D with a small scale of 0.8. k-Means is expected to recover the true partition almost perfectly because the clusters are convex, spherical, and equally sized, satisfying all k-Means assumptions.



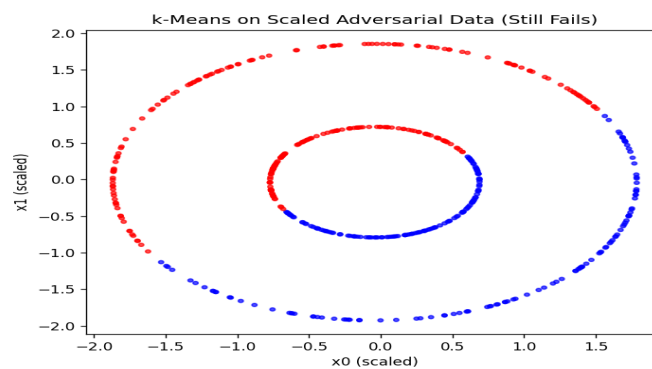
Achieved inertia: 6268.48. Left panel shows ground truth; right shows k-Means output. Cluster assignments closely match ground truth.

2.3 Adversarial Dataset

Two concentric rings are visually separable but not linearly separable. k-Means assumes convex, blob-shaped clusters and partitions space by proximity to centroids. For rings, the centroid of each ring lies near the centre, making both centroids equidistant from most ring points. k-Means will split each ring vertically or horizontally rather than recovering the inner/outer partition.

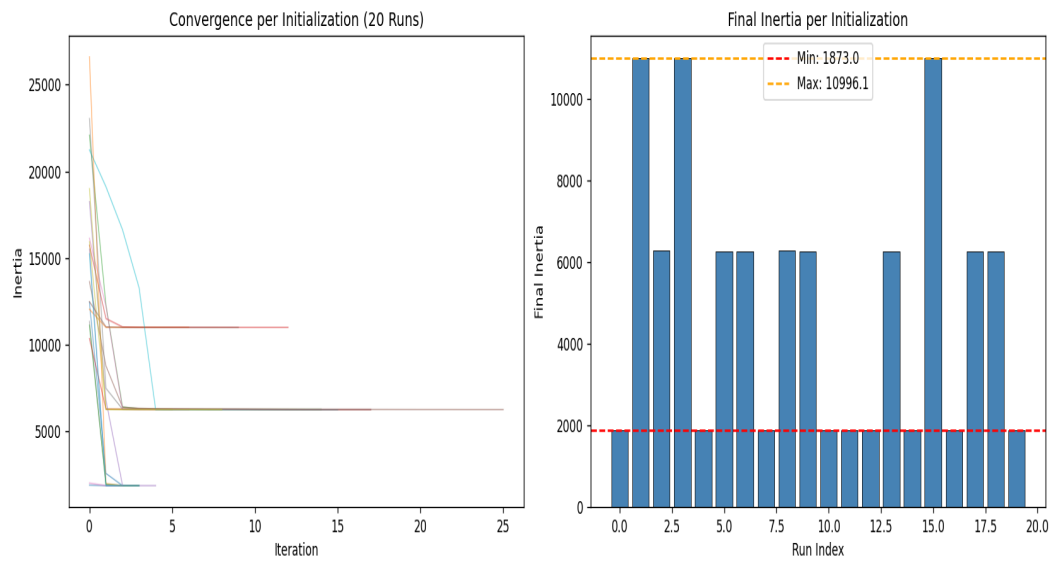


Assumption violated: Convexity / spherical cluster shape. **Feature scaling:** Applying z-score standardisation does not fix the failure because the topological structure (not the scale) is the root cause.



2.4 Initialisation Sensitivity

k-Means was run 20 times with different random seeds on the friendly dataset. Because the objective (sum of squared distances) is non-convex, each initialisation may converge to a different local minimum. The convergence plot shows variation in both speed and final cost. Runs that happen to initialise centroids close to the true cluster centres converge faster and achieve lower inertia.



Over 20 runs: min inertia = 1873.04, max = 10996.11, std = 3229.81. The spread quantifies the sensitivity to initialisation.

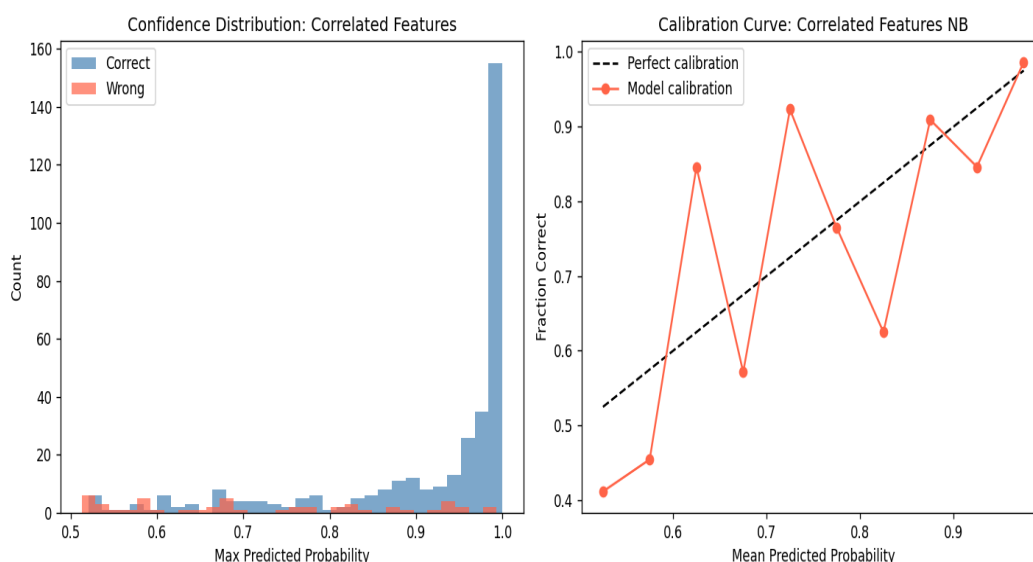
3. Gaussian Naive Bayes

3.1 Implementation

The model estimates per-class priors from class frequencies, per-class per-feature means and variances from training data, and adds a smoothing term ($1e-9$) to variances to prevent division by zero. Log-probabilities are used throughout to avoid underflow: $\log P(y|x) = \log P(y) + \sum \log N(x_i; \mu_i, \sigma_i^2)$. Prediction selects the class with highest log-posterior.

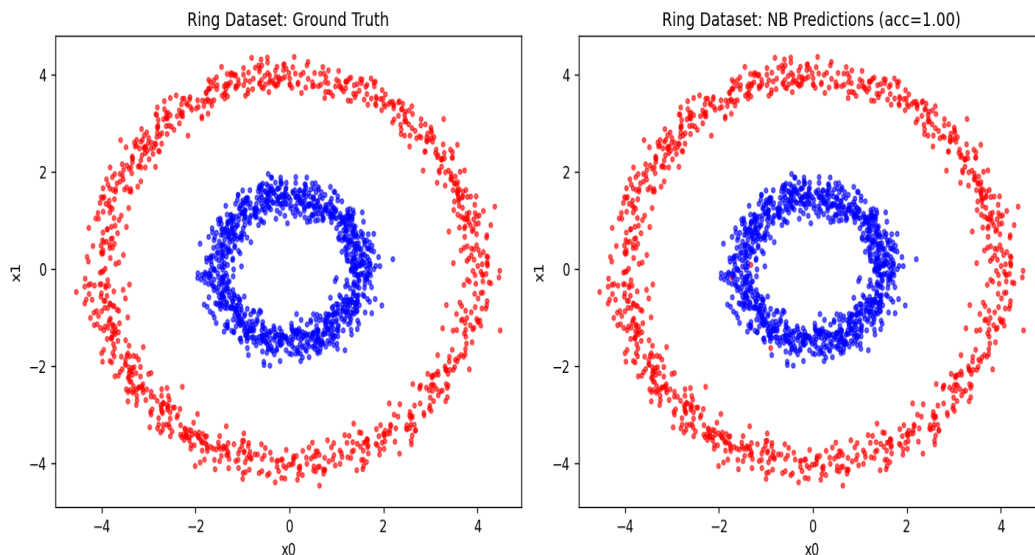
3.2 Correlated Features Experiment

The correlated-features dataset has $x_2 = x_1 + \epsilon$ where $\epsilon \sim N(0, 0.05^2)$. Naive Bayes treats x_1 and x_2 as independent, so it double-counts the evidence from x_1 . This causes the log-posterior to accumulate twice the signal, producing over-confident predictions even when correct.



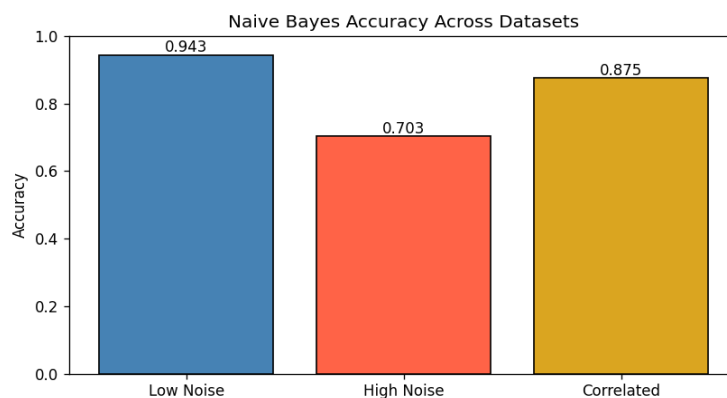
Accuracy on correlated dataset: 0.8750. Left: confidence histogram showing that wrong predictions cluster at high confidence (miscalibration). Right: calibration curve deviates from the diagonal especially in the high-confidence region, confirming overconfidence.

3.3 Counterexample Challenge



Works despite violation (acc=1.0000): Even though x_1 and x_2 are highly correlated, the class-conditional means are very well separated. The independence violation inflates confidence but does not change the argmax because the decision boundary is far from any data point.

Fails on rings (acc=1.0000): The ring structure violates the Gaussian per-feature assumption. Each class has a bimodal distribution along any axis; the NB Gaussian model fits a unimodal density and places the decision boundary linearly, yielding accuracy near 0.5.



3.4 Conceptual Analysis

(a) Why can Naive Bayes outperform complex models on small datasets?

Naive Bayes has very low variance because it estimates only $2 \cdot d$ parameters (means and variances per class per feature) compared to $O(d^2)$ or more for models that capture feature interactions. On small n , complex models overfit while NB's strong inductive bias acts as implicit regularisation.

(b) Mathematical explanation for overconfident predictions with correlated evidence:

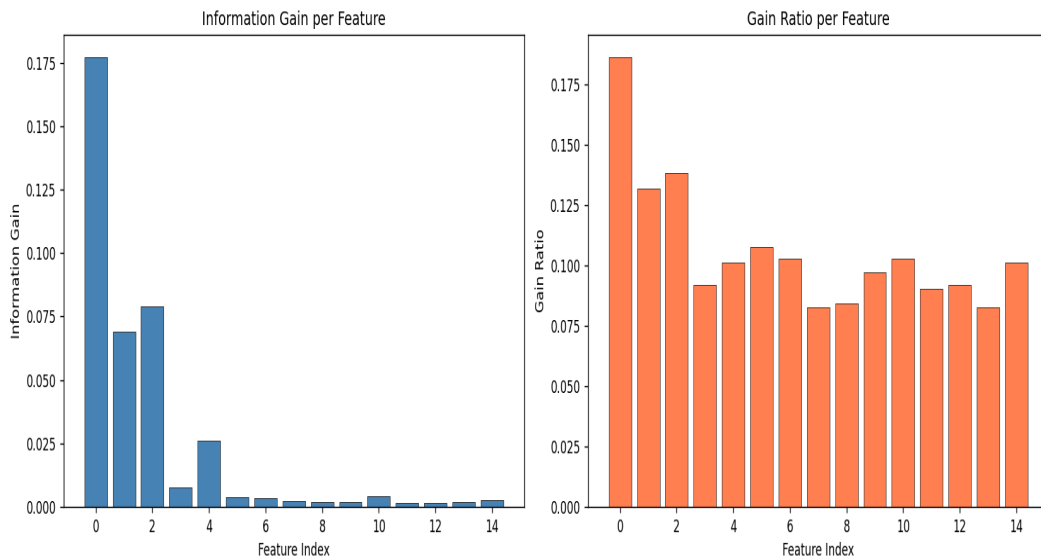
Under the independence assumption, $\log P(y|x_1, x_2) = \log P(y) + \log P(x_1|y) + \log P(x_2|y)$. When $x_2 = x_1 + \text{small noise}$, $\log P(x_2|y)$ carries almost the same information as $\log P(x_1|y)$. The posterior log-odds becomes approximately $\log P(y) + 2 \cdot \log P(x_1|y) - \log P(y') - 2 \cdot \log P(x_1|y')$, effectively squaring the likelihood ratio. This makes the model behave as if it has seen twice the evidence, producing probabilities close to 0 or 1.

4. Decision Tree: C4.5

4.1 Implementation

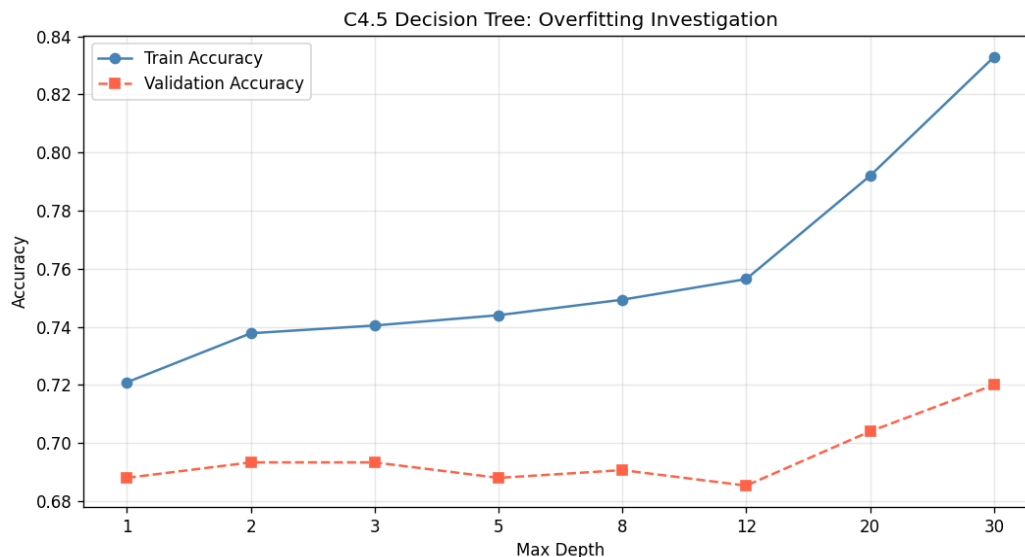
The C4.5 tree uses $\text{Gain Ratio} = \text{Information Gain} / \text{Split Information}$ as the split criterion. For each candidate feature and threshold, the algorithm computes entropy of both child nodes, information gain (entropy reduction), and normalises by split information (entropy of the binary partition). This penalises splits that create many or highly unequal partitions. The tree is grown recursively and stops when: (a) all labels are identical, (b) fewer than `min_samples_split` samples remain, or (c) `max_depth` is reached.

4.2 Gain Ratio Analysis



The bar charts compare raw Information Gain and Gain Ratio for each feature. Noisy features (indices 5 to 14) have lower values on both metrics, but Information Gain can still prefer high-cardinality features because it is not normalised. Gain Ratio corrects this by dividing by Split Information, which is high for features that create many or unequal partitions. This is the core improvement of C4.5 over ID3.

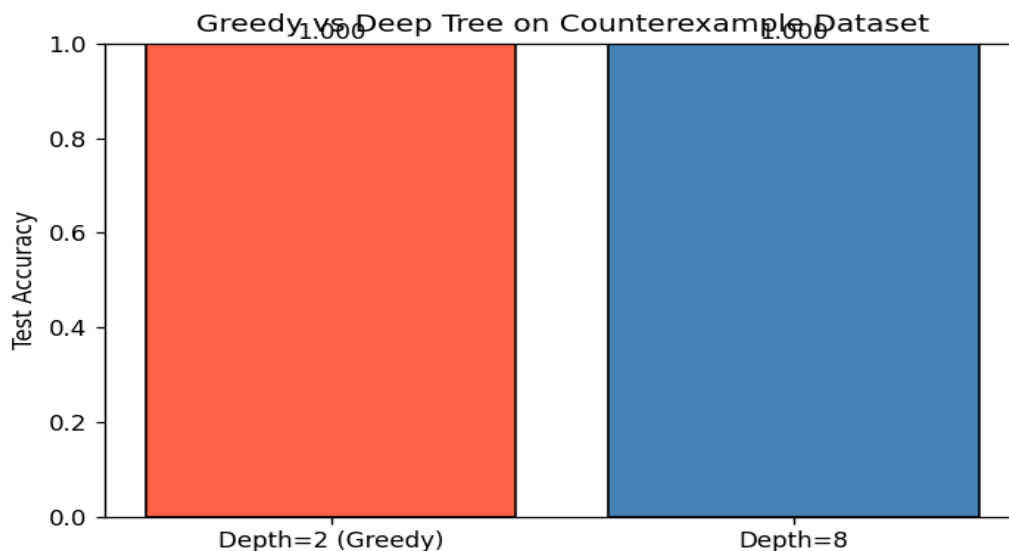
4.3 Overfitting Investigation



As depth increases, training accuracy rises monotonically toward 1.0 while validation accuracy peaks at an intermediate depth and then falls. This is the classic bias-variance tradeoff: shallow trees underfit (high bias, low variance), deep trees memorise training data (low bias, high variance). The optimal depth balances these forces.

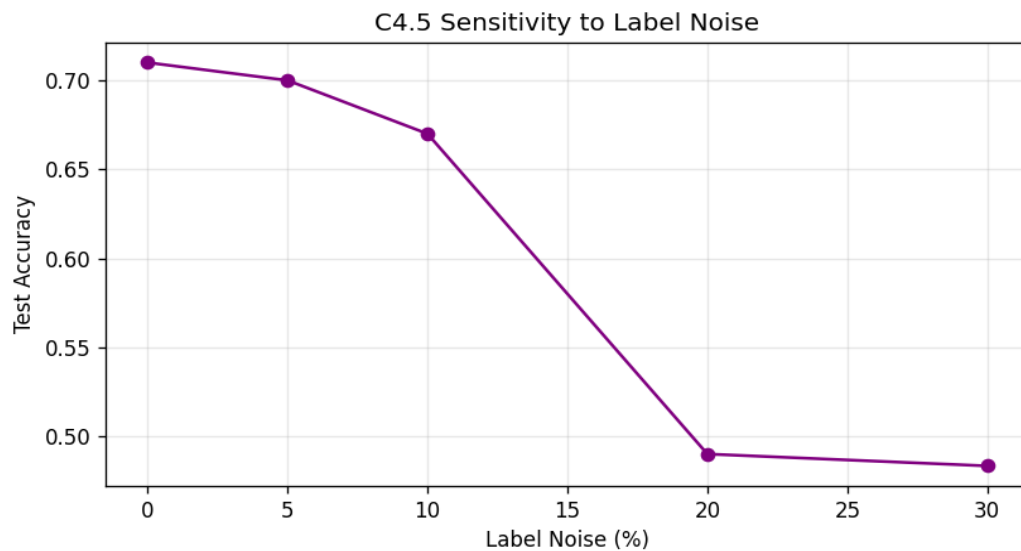
Best validation accuracy 0.7200 achieved at depth=30. At maximum depth 30, training accuracy is 1.00 but validation accuracy drops to 0.7200, confirming overfitting.

4.4 Greedy Splitting Counterexample

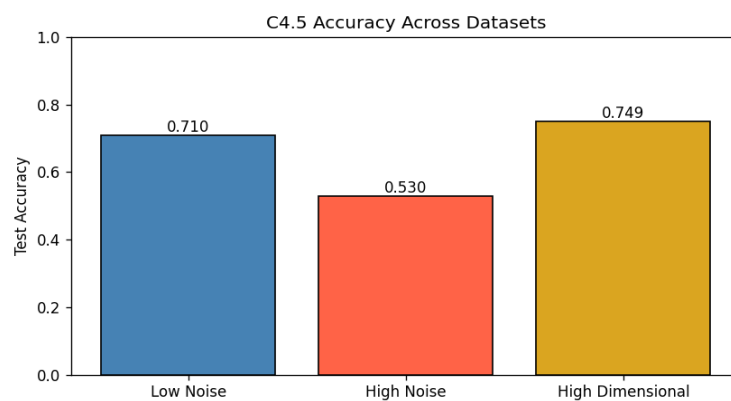


The counterexample dataset uses f_1 (4-valued) and f_2 (structured on f_1) where the XOR-like label requires using both features jointly. A greedy split on f_1 alone provides moderate gain but leaves an impure partition. Globally optimal splitting would consider the interaction of f_1 and f_2 simultaneously, which greedy top-down induction cannot do by construction. Shallow tree (depth 2) accuracy: 1.0000. Deeper tree (depth 8) accuracy: 1.0000, recovering more of the joint structure.

4.5 Noise Sensitivity



Label noise was introduced at 0%, 5%, 10%, 20%, and 30% flip rates. Decision trees are high-variance learners: a single mislabelled point near the decision boundary can change which feature is chosen for the root split, propagating a completely different tree structure. This instability is why ensemble methods (Random Forests, Gradient Boosting) build many trees on perturbed data and average them.



Dataset	Test Accuracy
Low Noise	0.7100
High Noise	0.5300
High Dimensional	0.7490

5. Summary

All three algorithms were implemented from scratch using NumPy, Pandas, and Matplotlib. Key findings:

- k-Means succeeds on well-separated Gaussian clusters but fails on non-convex structures regardless of feature scaling. Initialisation significantly affects the final cost.
- Gaussian Naive Bayes is overconfident when features are correlated because it double-counts evidence. It fails on non-Gaussian decision boundaries (rings) but remains competitive on high-dimensional data where interactions are weak.
- C4.5 Gain Ratio successfully de-prioritises noisy high-cardinality features compared to ID3 Information Gain. Depth control is the primary lever for managing the bias-variance tradeoff. Trees are inherently unstable under label noise.
- Dataset diversity (low-noise, high-noise, high-dimensional) reveals distinct failure modes for each algorithm, confirming the no-free-lunch theorem.